

Python for AI Beginners to Pro

Chapter 1: Python Basics for AI

1.1 Print, variables, and input

python

```
name = input("Enter your name: ")
```

```
age = int(input("Enter your age: "))
```

```
print("Name:", name)
```

```
print("Age:", age)
```

Exercise 1

Write a program that asks for your favorite AI field and prints it in a sentence.

Exercise 2

Take two numbers as input and print their sum, difference, product, and quotient.

Chapter 2: Data Types and Conditions

2.1 Data types

python

```
x = 10
```

```
y = 3.5
```

```
z = "AI"
```

```
a = True
```

```
print(type(x))
```

```
print(type(y))
```

```
print(type(z))
```

```
print(type(a))
```

2.2 Conditions

python

```
marks = int(input("Enter marks: "))
```

```
if marks >= 90:
```

```
    print("Excellent")
```

```
elif marks >= 75:
```

```
    print("Very Good")
```

```
elif marks >= 50:
```

```
    print("Pass")
```

```
else:
```

```
    print("Fail")
```

Exercise 1

Write a program to check whether a number is even or odd.

Exercise 2

Write a program to find the larger of two numbers.

Chapter 3: Loops

3.1 For loop

python

```
for i in range(5):
```

```
    print(i)
```

3.2 While loop

python

```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
    count += 1
```

Exercise 1

Print numbers from 1 to 20 using a loop.

Exercise 2

Print the multiplication table of 7.

Chapter 4: Strings and Lists

4.1 String operations

python

```
text = "Python for AI"
```

```
print(text.upper())
```

```
print(text.lower())
```

```
print(text.replace("AI", "Artificial Intelligence"))
```

```
print(text.split())
```

4.2 Lists

python

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[0])
```

```
numbers.append(60)
```

```
print(numbers)
```

Exercise 1

Take a sentence and count how many words it contains.

Exercise 2

Create a list of 5 student marks and find the average.

Chapter 5: Functions

5.1 Basic function

python

```
def greet(name):
```

```
    return f'Hello, {name}!'
```

```
print(greet("AI Learner"))
```

5.2 Function with calculation

python

```
def square(num):
```

```
return num * num
```

```
print(square(8))
```

Exercise 1

Write a function that returns the factorial of a number.

Exercise 2

Write a function that checks whether a number is prime.

Chapter 6: Dictionaries and File Handling

6.1 Dictionary

```
python
```

```
student = {  
    "name": "Ravi",  
    "age": 21,  
    "branch": "AI"  
}
```

```
print(student["name"])
```

6.2 File writing

```
python
```

```
with open("note.txt", "w") as file:  
    file.write("Python for AI")
```

6.3 File reading

```
python
```

```
with open("note.txt", "r") as file:  
    content = file.read()  
    print(content)
```

Exercise 1

Create a dictionary for 3 students and print only their names.

Exercise 2

Write and read a text file containing your AI study goals.

Chapter 7: OOP for AI

7.1 Class and object

python

class Model:

```
def __init__(self, name, accuracy):  
    self.name = name  
    self.accuracy = accuracy
```

```
def show(self):  
    print(self.name, self.accuracy)
```

```
m1 = Model("Spam Detector", 92)
```

```
m1.show()
```

Exercise 1

Create a Student class with name, roll number, and marks.

Exercise 2

Create a Car class with a method to display details.

Chapter 8: NumPy for AI

8.1 Arrays

python

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
print(arr.mean())
```

```
print(arr.max())
```

8.2 Two-dimensional array

python

```
matrix = np.array([[1, 2], [3, 4], [5, 6]])
```

```
print(matrix)
```

```
print(matrix.shape)
```

Exercise 1

Create a NumPy array of 10 numbers and find mean, min, and max.

Exercise 2

Create a 3x3 matrix and print its shape.

Chapter 9: Pandas for Data Handling

9.1 DataFrame

python

```
import pandas as pd
```

```
data = {  
    "Name": ["A", "B", "C"],  
    "Marks": [85, 90, 78]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

9.2 Read CSV

python

```
df = pd.read_csv("students.csv")
```

```
print(df.head())
```

Exercise 1

Create a DataFrame of 5 employees with name, age, and salary.

Exercise 2

Load any CSV file and print the first 5 rows.

Chapter 10: Data Visualization

10.1 Line plot

python

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [10, 15, 13, 18]
```

```
plt.plot(x, y)
```

```
plt.title("Simple Line Plot")
```

```
plt.show()
```

10.2 Bar chart

python

```
names = ["A", "B", "C"]
```

```
marks = [80, 70, 90]
```

```
plt.bar(names, marks)
```

```
plt.title("Marks Chart")
```

```
plt.show()
```

Exercise 1

Draw a bar chart of your subject marks.

Exercise 2

Draw a line chart showing 5 days of temperature values.

Chapter 11: Data Cleaning

11.1 Missing values

python

```
import pandas as pd
```

```
data = {  
    "Name": ["A", "B", "C"],  
    "Marks": [85, None, 78]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df.isnull())
```

```
print(df.fillna(0))
```

Exercise 1

Create a DataFrame with missing values and fill them using mean or zero.

Exercise 2

Drop rows with missing values from a dataset.

Chapter 12: Machine Learning Basics

12.1 Train a simple model

python

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
import pandas as pd
```

```
data = {  
    "Hours": [1, 2, 3, 4, 5],  
    "Score": [20, 30, 40, 50, 60]  
}
```

```
df = pd.DataFrame(data)
```


BrainStack

```
X = df[["Hours"]]
```

```
y = df["Score"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
print(model.predict(X_test))
```

Exercise 1

Change the dataset and predict exam marks from study hours.

Exercise 2

Try predicting house price using area as input.

Chapter 13: Classification

13.1 Simple classifier

```
python
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
X = [[1], [2], [3], [6], [7], [8]]
```

```
y = [0, 0, 0, 1, 1, 1]
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X, y)
```

```
print(model.predict([[4]]))
```

Exercise 1

Create a classifier for pass/fail based on marks.

Exercise 2

Create a classifier for yes/no based on input feature values.

Chapter 14: Model Evaluation

14.1 Accuracy

python

```
from sklearn.metrics import accuracy_score
```

```
y_true = [0, 1, 1, 0, 1]
```

```
y_pred = [0, 1, 0, 0, 1]
```

```
print(accuracy_score(y_true, y_pred))
```

Exercise 1

Test accuracy using your own actual and predicted values.

Exercise 2

Build a confusion matrix for a binary classification task.

Chapter 15: NLP Basics

15.1 Simple text processing

python

```
text = "Python is great for AI"
```

```
words = text.lower().split()
```

```
print(words)
```

15.2 Word frequency

python

```
from collections import Counter
```

```
text = "ai python ai machine learning python"
```

```
words = text.split()
```

```
count = Counter(words)
```

```
print(count)
```

Exercise 1

Count the frequency of words in a paragraph.

Exercise 2

Write a program to remove punctuation from text.

Chapter 16: Mini AI Projects

Project 1: Spam detector

python

```
messages = ["win money now", "hello friend", "free offer", "good morning"]
```

```
labels = [1, 0, 1, 0]
```

Use this as a starting point and build a text classifier.

Project 2: House price prediction

python

use linear regression

Project 3: Student result predictor

python

use marks and attendance as inputs

Exercise 1

Build a small chatbot that replies to greetings.

Exercise 2

Build a sentiment detector for positive and negative words.

What you can build after this

After completing these chapters, you should be able to build beginner AI projects such as spam detection, prediction models, simple chatbots, text analysis tools, and small data apps. As you continue practicing, you can move into deep learning, computer vision, and more advanced AI projects.

Mini Project 1: Rule-Based AI Chatbot

What it does

This chatbot replies to simple user inputs like greetings, name questions, and help requests. It is a good first AI-style project because it teaches text handling, conditional logic, and basic natural language matching.

Code

python

```
def chatbot_response(user_input):
    text = user_input.lower()

    greetings = ["hi", "hello", "hey", "good morning", "good evening"]
    thanks = ["thank you", "thanks", "thx"]
    help_words = ["help", "support", "problem", "issue"]

    if any(word in text for word in greetings):
        return "Hello! How can I help you today?"
    elif "your name" in text:
        return "I am a simple Python AI chatbot."
    elif "my name" in text:
        return "Nice to meet you!"
    elif any(word in text for word in thanks):
        return "You're welcome!"
    elif any(word in text for word in help_words):
        return "Tell me your problem, and I will try to guide you."
    elif "bye" in text or "exit" in text:
        return "Goodbye! Have a great day."
    else:
        return "I am still learning. Can you ask something simple?"

print("AI Chatbot Started. Type 'exit' to stop.\n")
while True:
    user = input("You: ")
    response = chatbot_response(user)
    print("Bot:", response)
```

BrainStack

```
if user.lower() in ["bye", "exit"]:
    break
```

How it works

The chatbot checks the user input against lists of words such as greetings, thanks, and help-related terms. If a match is found, it returns a fixed response using `if`, `elif`, and `any()`.

What you learn

- String handling.
- Functions.
- Lists.
- Condition checking.
- Basic chatbot logic.

How to improve it

You can later add memory, user names, more keyword groups, or even connect it to a machine learning or LLM-based system.

AI Chatbot Started. Type "exit" to stop.

You: hi

Bot: Hello! How can I help you today?

You: what is your name?

Bot: I am a simple Python AI chatbot.

You: thanks

Bot: You're welcome!

You: bye

Bot: Goodbye! Have a great day.

Mini Project 2: Spam Message Detector

What it does

This program classifies a message as spam or not spam based on simple keyword scoring. It is a beginner version of text classification and is a great introduction to AI-style decision making.

Code

```
python
spam_keywords = [
    "win", "free", "cash", "offer", "urgent",
    "lottery", "prize", "claim", "bonus", "click"
]

def detect_spam(message):
    text = message.lower()
    score = 0

    for word in spam_keywords:
        if word in text:
            score += 1

    if score >= 2:
        return "Spam"
    else:
        return "Not Spam"

print("Spam Detector Started\n")

while True:
    msg = input("Enter a message (type 'exit' to stop): ")
    if msg.lower() == "exit":
        break

    result = detect_spam(msg)
    print("Result:", result)
```

How it works

The program stores common spam-related words in a list. It checks whether those words appear in the message and counts matches; if the score is high enough, the message is labeled spam.

What you learn

- Text preprocessing.
- Keyword matching.
- Looping through lists.
- Simple classification logic.
- AI-style decision rules.

How to improve it

You can make it better by adding a dataset, using Scikit-learn, or training a real machine learning classifier with labeled spam and ham messages.

Spam Detector Started

```
Enter a message (type 'exit' to stop): Hey, are we still meeting tomorrow?  
Result: Not Spam
```

```
Enter a message (type 'exit' to stop): Congratulations! You won a free prize  
Result: Spam
```

```
Enter a message (type 'exit' to stop): Claim your cash bonus now, click here!  
Result: Spam
```

```
Enter a message (type 'exit' to stop): Are you coming to the party tonight?  
Result: Not Spam
```

```
Enter a message (type 'exit' to stop): Urgent offer! Win cash now!!!  
Result: Spam
```

```
Enter a message (type 'exit' to stop): exit
```